2nd Slide Set Operating Systems

Prof. Dr. Christian Baun

Frankfurt University of Applied Sciences
(1971–2014: Fachhochschule Frankfurt am Main)
Faculty of Computer Science and Engineering
christianbaun@fb2.fra-uas.de

Learning Objectives of this Slide Set

Operating systems can be classified according to different criteria

The most important distinction criteria are discussed in this slide set

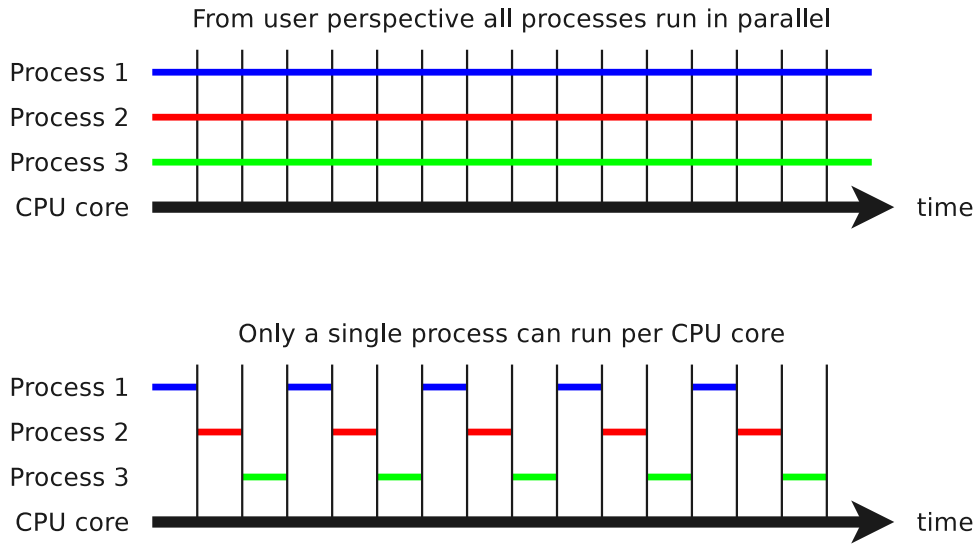
- At the end of this slide set, you know/understand...
  - the difference between **singletasking** and **multitasking**
  - the difference between **single-user** and **multi-user**
  - the reason for the **memory address length**
  - what **real-time operating systems** are
  - what **distributed operating systems** are
  - the different **kernel architectures**
    - **Monolithic kernel**, **Microkernel**, **Hybrid kernel**
  - the **structure (layers)** of operating systems
- what the steps of the **boot process** (bootstrap) are

Exercise sheet 2 repeats the contents of this slide set which are relevant for these learning objectives

Singletasking and Multitasking

- **Singletasking**
  - At any given moment, only a single program is executed
  - Multiple started programs are executed **one after the other**

- **Multitasking**
  - Multiple programs can be executed **at the same time** (with multiple CPUs/Cores) or **quasi-parallel**



Task, process, job,...

The term **task** is equivalent to **process** or from the user's point of view, **job**

Why Multitasking?

We already know...

- With **multitasking**, multiple processes are executed concurrently
- The processes are activated alternately at short intervals
  $\Longrightarrow$ For this reason, **the execution appears simultaneous**
- Drawback: Switching from one process to another one causes **overhead**

- Processes often need to wait for external events
  - External events may be user inputs, input/output operations of peripheral devices, or simply waiting for a message from another program
  - With multi-tasking, processes which wait for incoming e-mails, successful database operations, data written to the HDD, or something similar can be placed in the background
    - This way, other **processes can be executed sooner**
- The program switching, which is required to implement the quasi-parallel execution, causes overhead
  - **The overhead is rather small compared with the speedup**

Single-user and Multi-user

- **Single-User**
  - The computer can only be used by a single user at any point in time
- **Multi-User**
  - Multiple users can work simultaneously with the computer
    - Users share the system resources (as fair as possible)
    - Users must be identified (via passwords)
    - Access to data/processes of other users must be prevented

	Single-User	**Multi-User**
Singletasking	CP/M 2.2, MS-DOS, PC-DOS, DR-DOS, GEOS, Palm OS, FreeDOS	impossible
Multitasking	OS/2, Windows 3x/95/98, BeOS, MacOS 8x/9x, AmigaOS, Risc OS, ArcaOS, eComStation, KolibriOS, MenuetOS, Plan 9, Haiku, FreeRTOS, iOS (iPhone), iPadOS (in normal operation mode)	Linux/UNIX, Minix, MacOS X, Server editions of the Windows NT family, OpenVMS, Android, ChromeOS, FreeBSD, NetBSD, DragonFly, OpenIndiana, Solaris, ReactOS, iPadOS (with Shared iPad)

- Desktop/Workstation versions of Windows NT/XP/Vista/7/8/10/11 are only **half multi-user operating systems**
  - Different users can work with the system only one after the other, but the data and processes of the different users are protected from each other

8/16/32/64 bit Operating Systems

- The bit number indicates the **memory address length**, with which the operating system works internally
  - The number of memory units, an operating system can address, is limited by the address space, which is limited by the address bus $\Longrightarrow$ slide set 3
- **8 bit operating systems** can address 2^8 memory units
  - e.g. GEOS, Atari DOS, Contiki
- **16 bit operating systems** can address 2^{16} memory units
  - e.g. MS-DOS, Windows 3.x, OS/2 1.x

Bill Gates (1989)

"We will never make a 32-bit operating system."

- **32 bit operating systems** can address 2^{32} memory units
  - e.g. Windows 95/98/NT/Vista/7/8/10, OS/2 2/3/4, eComStation, Linux, BeOS, MacOS X (until 10.7)
- **64 bit operating systems** can address 2^{64} memory units
  - e.g. Linux (64 bit), Windows 7/8/10/11 (64 bit), MacOS X (64 bit)

Real-Time Operating Systems

- Are multitasking operating systems with additional real-time functions for the compliance of time conditions
- Essential criteria of real-time operating systems:
  - **Response time**
  - Meet **deadlines**
- Different priorities are taken into account so that important processes are executed within certain time limits
- 2 types of real-time operating systems exist:
  - **Hard real-time operating systems**
  - **Soft real-time operating systems**
- Modern desktop operating systems can **guarantee** soft real-time behavior for processes with high priority
  - Because of the unpredictable time behavior due to swapping, hardware interrupts, etc. **hard real-time behavior** cannot be guaranteed

Hard and Soft Real-Time Operating Systems

- **Hard real-time operating systems**
  - Deadlines must be strictly met
  - Delays cannot be accepted under any circumstances
  - Delays lead to disastrous consequences and high cost
  - Results are useless if they are achieved too late
  - Application examples: Welding robot, reactor control, Anti-lock braking system (ABS), aircraft flight control, monitoring systems of an intensive care unit
- **Soft real-time operating systems**
  - Certain tolerances are allowed
  - Delays cause acceptable costs
  - Typical applications: Telephone system, parking ticket vending machine, ticket machine, multimedia applications such as audio/video on demand

Application Areas of Real-Time Operating Systems

- Typical application areas of real-time operating systems:
  - Embedded systems and IoT devices
  - Industrial monitoring systems
  - Robots
- Examples of real-time operating systems:
  - QNX
  - VxWorks
  - LynxOS
  - RTLinux
  - FreeRTOS
  - Zephyr
  - Apache NuttX
  - Real-Time Operating System for Multiprocessor Systems (RTEMS)
  - Symbian (outdated)
  - Windows CE (outdated)

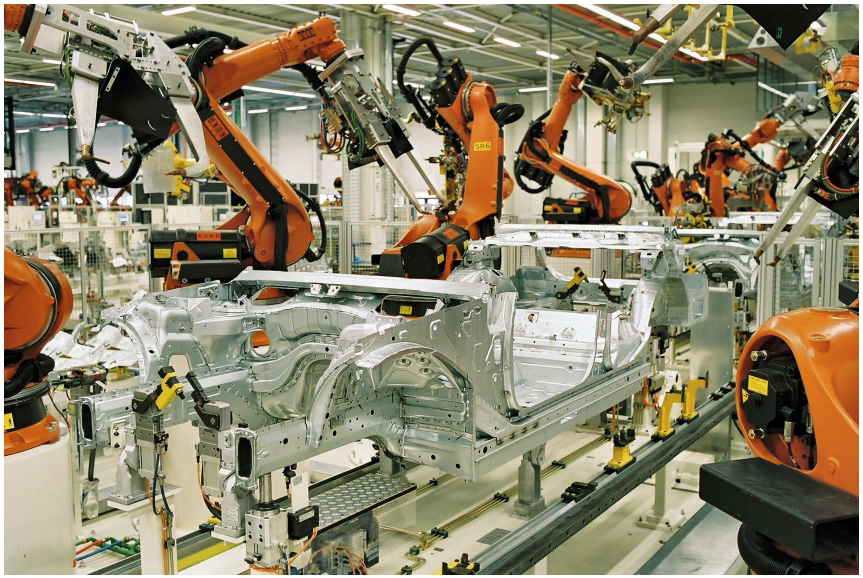


Image source: BMW Werk Leipzig (CC-BY-SA 2.0)

Have some Fun with the QNX Demo Disc from 1999...

web.archive.org/web/20011019174050/www.qnx.com/demodisk/

THE INCREDIBLE 1.44M DEMO

build a more reliable world™

Download QNX 4 Demo | Extend OS functionality | Troubleshooting | Demo Home

Find

- How we did it
- What people are saying
- Send me info
- Comments
- Real World applications
- Ordering QNX products

QNX web site

- +QNX Home
- +Get QNX
- +Contact QNX
- +Search

Product info

- +QNX Realtime Platform
- +QNX 4 RTOS
- +QNX Neutrino
- +Photon microGUI
- +PhAB
- +Internet Suite
- +IAT

more ...

THE INCREDIBLE 1.44M DEMO

Version 4 is here!

This single **bootable** floppy contains:

realtime OS	dialer
GUI	TCP/IP
browser	sample apps
server	and much more ...

Surf the web. Serve HTML pages. Extend the OS - on the fly ...

Note:

This is a demo of the QNX 4 RTOS. To download the new QNX realtime platform, visit get.qnx.com.

Create your own demo

All you need is a 1.44M floppy! Just insert your disk, click on "Create a demo," and follow instructions.

Once you've downloaded the 1.44M QNX Demo, you can:

- Surf the web - Use your IP address to connect, and get ready to surf!
- Generate dynamic HTML - Even diskless systems can generate HTML pages in real time.
- Extend the OS - Download drivers on the fly - without rebooting.

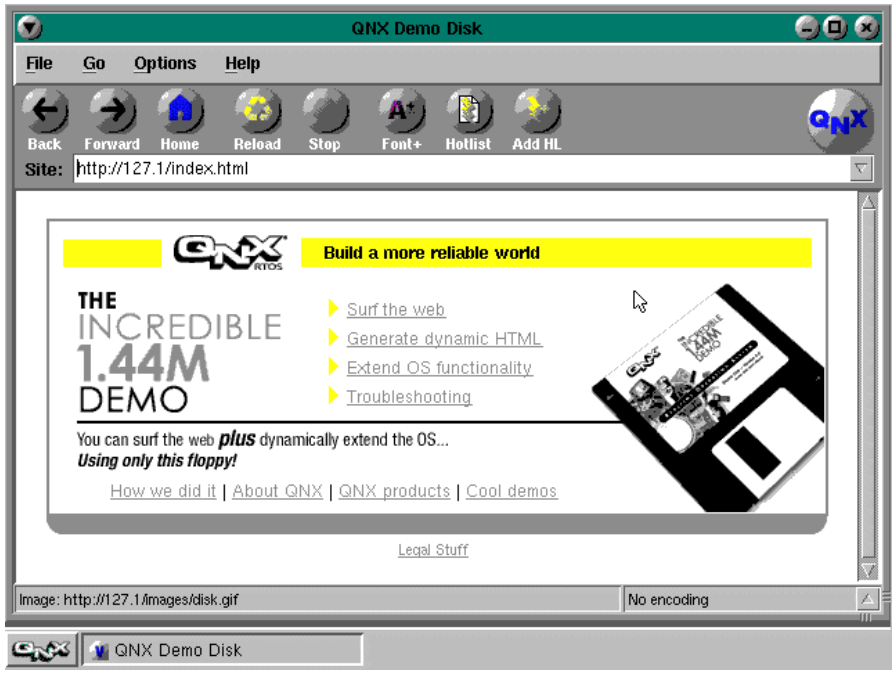


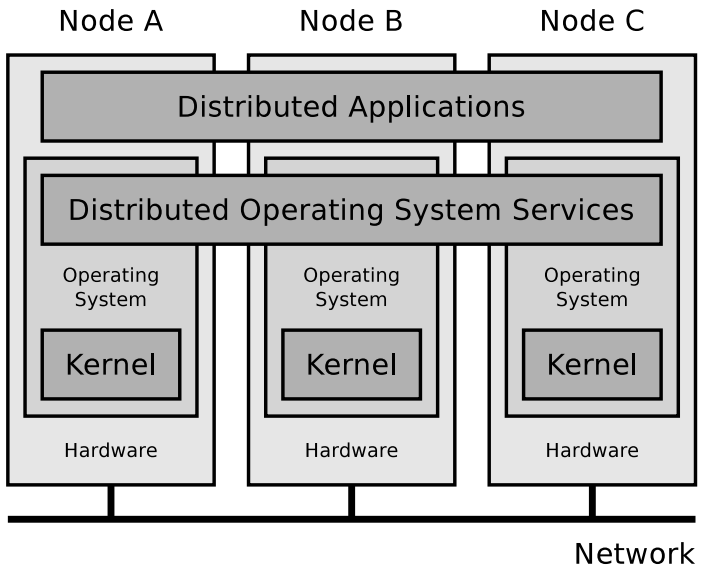
Image source:
`http://toastytech.com/guis/qnxdemo.html`

Impressive video of the demo disc:
`https://www.youtube.com/watch?v=K_VlI6IBEJ0`

Distributed Operating Systems

- Distributed system
- Controls processes on multiple computers of a cluster
- The individual computers remain transparently hidden from the users and their applications
  - The system appears as a single large computer
    - **Single System Image** principle

- The principle of distributed operating systems is dead!
- However, during the development of some distributed operating systems some interesting technologies have been developed and applied for the first time
- Some of these technologies are still relevant today



Distributed Operating Systems (1/3)

- **Amoeba**
  - Mid-1980s to mid-1990s
  - Andrew S. Tanenbaum (Free University of Amsterdam)
  - The programming language Python was developed for Amoeba

`http://www.cs.vu.nl/pub/amoeba/`

The Amoeba Distributed Operating System. *A. S. Tanenbaum, G. J. Sharp.* `http://www.cs.vu.nl/pub/amoeba/Intro.pdf`

- **Inferno**
  - Based on the UNIX operating system Plan 9
  - Bell Laboratories
  - Applications are programmed in the programming language Limbo
    - Similar to Java, Limbo produces bytecode, which is executed by a virtual machine
  - Minimal hardware requirements
    - Requires only 1 MB of main memory

`http://www.vitanuova.com/inferno/index.html`

Distributed Operating Systems (2/3)

- **Rainbow**
  - Universität Ulm
  - Concept of a common memory to implement an uniform address space, in which all computers in the cluster can store and access objects
    - For applications, it is transparent, on which computer in the cluster, objects are physically located
    - Applications can access desired objects via uniform addresses from any computer
    - If the object is physically located in the memory of a remote computer, Rainbow does the transmission and local deployment to the requesting computer in an automated and transparent way

Rainbow OS: A distributed STM for in-memory data clusters. *Thilo Schmitt, Nico Kämmer, Patrick Schmidt, Alexander Weggerle, Steffen Gerhold, Peter Schulthess*. MIPRO 2011

Distributed Operating Systems (3/3)

- **Sprite**
  - University of California, Berkeley (1984-1994)
  - Connects workstations in a way that they appear to users as a single time-shared system
  - The parallel version of `make`, called `pmake` was developed for Sprite

The Sprite Network Operating System. 1988. `http://www.research.ibm.com/people/f/fdouglis/papers/sprite.pdf`

A Brief Retrospective on the Sprite Network Operating System

`http://www.stanford.edu/~ouster/cgi-bin/spriteRetrospective.php`

The Sprite Operating System

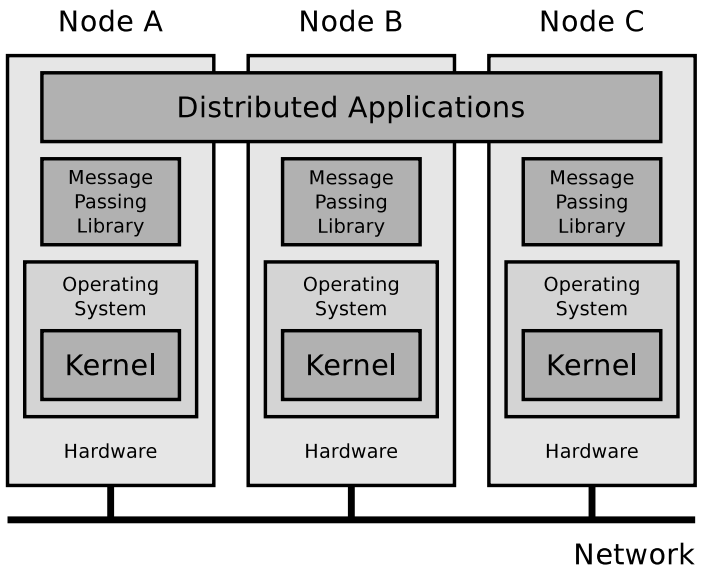
`https://github.com/OSPreservProject/sprite`

The first new Sprite cluster in 30 years (probably)

`https://www.reddit.com/r/vintageunix/comments/1trfb4y/the_first_new_sprite_cluster_in_30_years_probably/`

Distributed Operating Systems – Situation Today

- The concept did not gain acceptance
  - Distributed operating systems never left research projects state
  - Established operating systems have never been replaced
- For developing cluster applications, libraries exist, which provide hardware-independent **message passing**
  - Message passing communication is based on message exchange
  - Popular message passing systems:
    - **Message Passing Interface** (MPI)
      $\Longrightarrow$ standard solution
    - **Parallel Virtual Machine** (PVM)
      $\Longrightarrow$ †

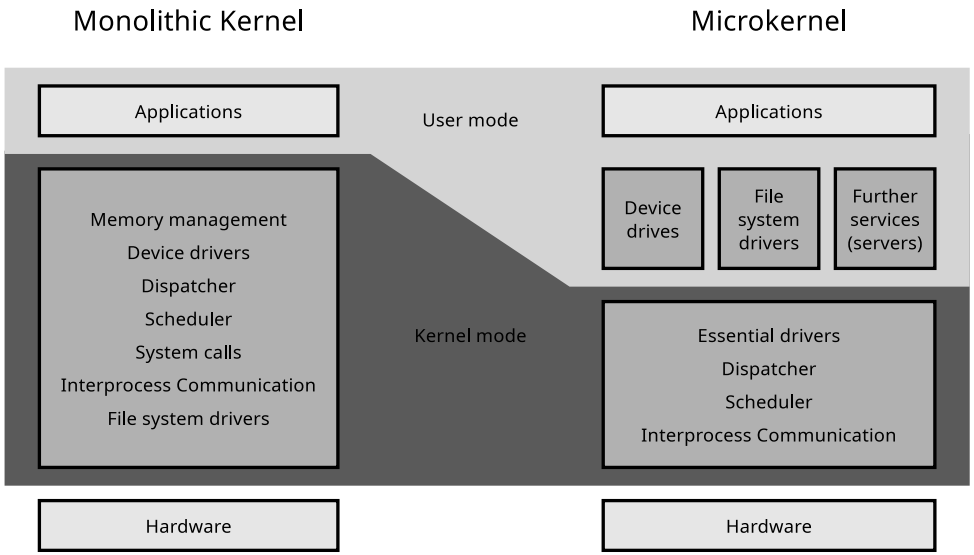


MPI tutorials

`http://mpitutorial.com/tutorials/`

Kernel Architectures

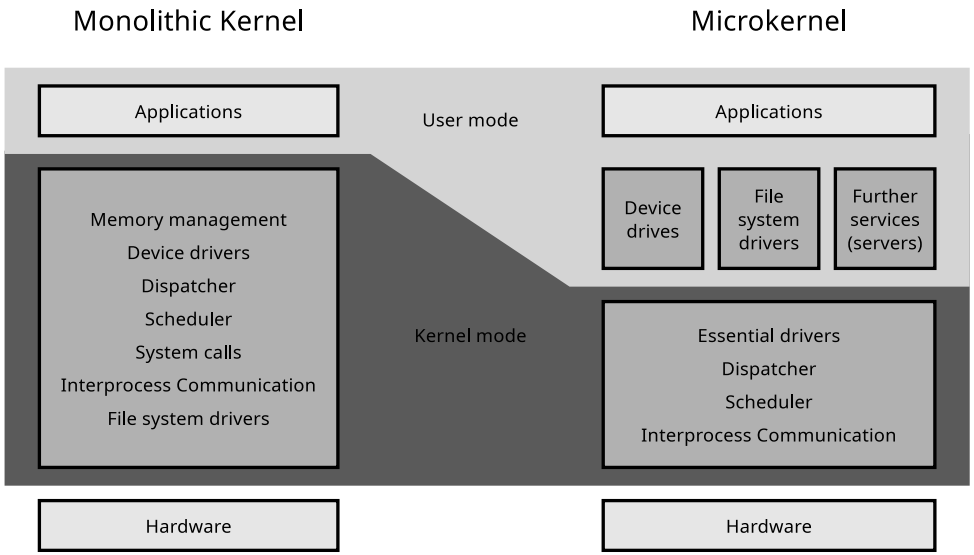
- The **kernel**...
  - contains the essential functions of the operating system and
  - is the interface to the hardware



- Different kernel architectures exist
  - They differ in which functions are **inside the kernel** and which functions are **outside the kernel** as services (servers)
- Functions in the kernel have full hardware access (**kernel mode**)
- Functions outside the kernel can only access their virtual memory (**user mode**)
  $\Longrightarrow$ slide set 5

Monolithic Kernels (1/2)

- Contain functions for...
  - memory management
  - process management
  - interprocess communication
  - hardware management (drivers)
  - file systems

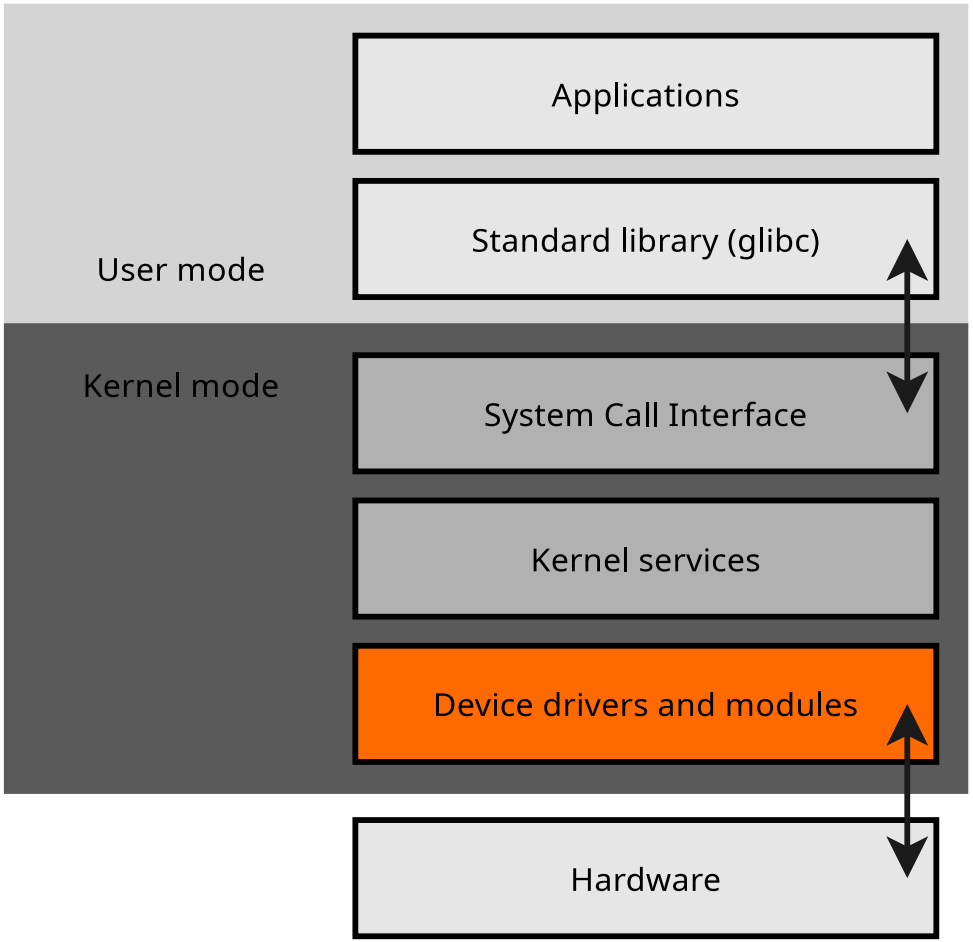


- Advantages:
  - Fewer context switching as with microkernels $\Longrightarrow$ better performance
  - Grown stability
    - Microkernels are usually not more stable compared with monolithic kernels
- Drawbacks:
  - Crashed kernel components can not be restarted separately and may cause the entire system to crash
  - Kernel extensions cause a high development effort, because for each compilation of the extension, the complete kernel needs to be recompiled

Monolithic Kernels (2/2)

Examples of operating systems with monolithic kernels

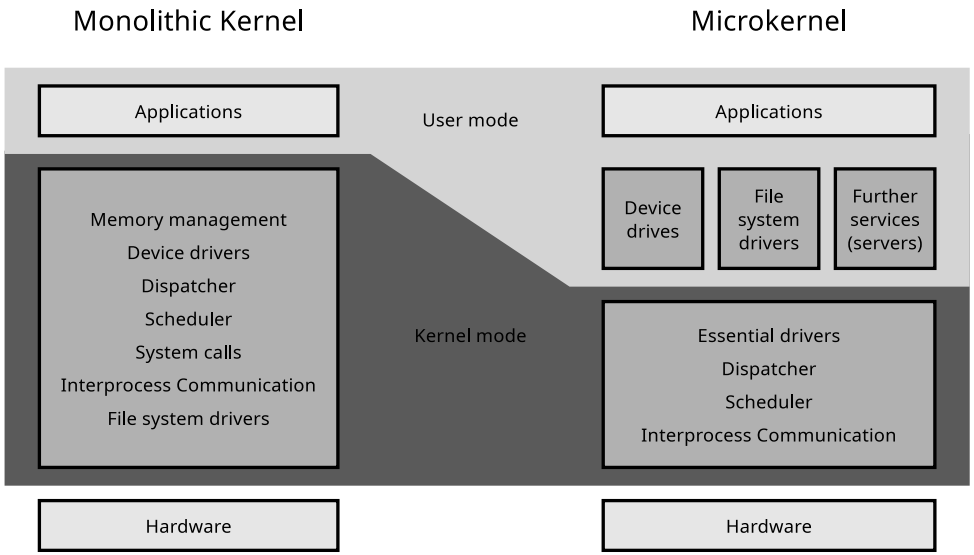
Linux, BSD, MS-DOS, FreeDOS, Windows 95/98/ME, MacOS (until 8.6), OS/2



- Linux is the most popular modern operating system with a monolithic kernel
- It is possible to outsource drivers of the **Linux kernel** into modules
  - However, the modules are executed in *kernel mode* and not in the *user mode*
    - Therefore, the Linux kernel is a monolithic kernel

Microkernels (1/2)

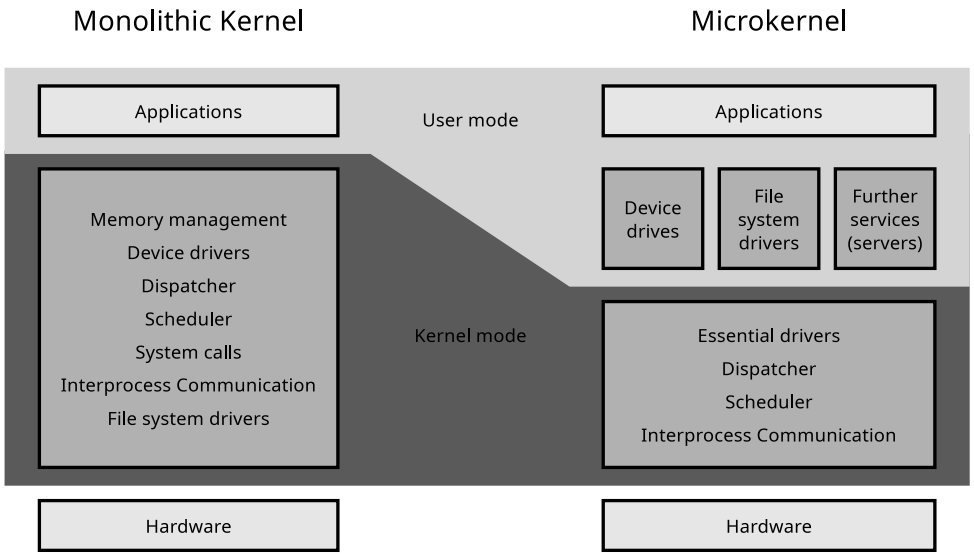
- The kernel contains only...
  - essential functions for memory management and process management
  - functions for process synchronization and interprocess communication
  - essential drivers (e.g. for system start)



- Device drivers, file systems, and services (servers) are located outside the kernel and run equal to the user applications in user mode

Microkernels (2/2)

- Advantages:
  - Components can be exchanged easily
  - Best stability and security in theory
    - Reason: Fewer functions run in kernel mode



- Drawbacks:
  - Slower because of more context switches
  - Development of a new (micro)kernel is a complex task

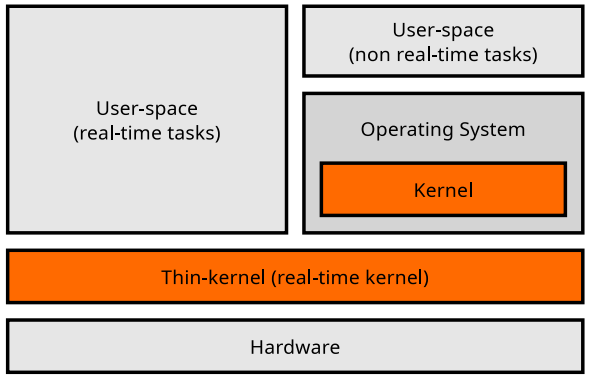
Examples of operating systems with microkernels

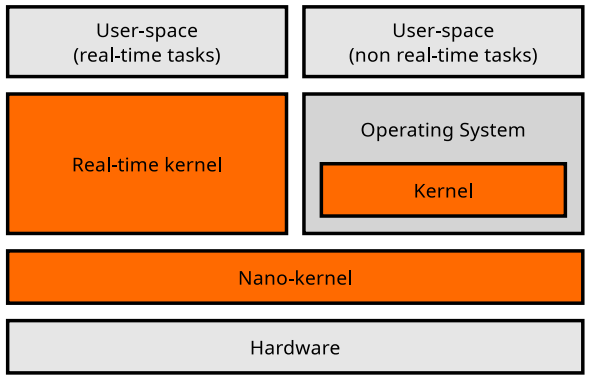
AmigaOS, MorphOS, Tru64, QNX Neutrino, Symbian OS, GNU HURD (see slide 24)

The success of the micro-kernel systems, which was forecasted in the early 1990s, did not happen $\Longrightarrow$ Discussion of Linus Torvalds vs. Andrew S. Tanenbaum (1992) $\Longrightarrow$ see slide 23

Kernel-Architectures of Real-Time Operating Systems

- **Thin kernel** (similar to a Microkernel)
  - The operating system kernel itself runs as a process with lowest priority in the background
  - The RT kernel does the scheduling
    - It is an abstraction interface between hardware and Linux kernel
  - Real-time processes have the highest priority $\Longrightarrow$ minimum reaction time (latency)
- **Nano kernel** (similar to a Microkernel too)
  - In addition to the RT kernel, several kernels of other operating systems may be executed
  - A nano kernel is similar to a Type-1-Hypervisor ($\Longrightarrow$ slide set 10)
- **Pico kernel, Femto kernel, Atto kernel**
  - Marketing buzz-words to emphasize the smallness of RT kernels





Source: Anatomy of real-time Linux architectures (2008): `http://www.ibm.com/developerworks/library/l-real-time-linux/`

Hybrid Kernels / Macrokernels

- Tradeoff between monolithic kernels and microkernels
  - They contain some components for performance reasons, which are never located inside microkernels
- It is not specified which additional components are located inside hybrid kernels
- Windows NT 4 indicates advantages and drawbacks of hybrid kernels
  - The kernel of Windows NT 4 contains the Graphics Device Interface
    - Advantage: Increased performance
    - Drawback: Buggy graphics drivers cause frequent crashes

Source: **MS Windows NT Kernel-mode User and GDI White Paper**. `https://technet.microsoft.com/library/cc750820.aspx`

- Advantage:
  - Better performance as with microkernels because fewer context switching
  - The stability is (theoretically) better as with monolithic kernels

Examples of operating systems with hybrid kernels

Windows NT family since NT 3.1, ReactOS, MacOS X, BeOS, ZETA, Haiku, Plan 9, DragonFly BSD

Linus Torvalds vs. Andrew Tanenbaum (1992)

Image Source: unknown



- August 26th 1991: Linus Torvalds announces the Linux project in the newsgroup `comp.os.minix`
  - September 17th 1991: First internal release (0.01)
  - October 5th 1991: First official release (0.02)
- 29. Januar 1992: Andrew S. Tanenbaum posts in the Newsgroup `comp.os.minix`: "**LINUX is obsolete**"
  - Linux has a monolithic kernel $\Longrightarrow$ step backwards
  - Linux is not portable, because it is optimized for the 80386 CPU and this architecture will soon be replaced by RISC CPUs (fail!)

This was followed by an intense and emotional several-day discussion about the advantages and drawbacks of monolithic kernel, microkernels, software portability and free software

A. Tanenbaum (30. January 1992): "*I still maintain the point that designing a monolithic kernel in 1991 is a fundamental error. Be thankful you are not my student. You would not get a high grade for such a design :-)*". Source: `http://www.oreilly.com/openbook/opensources/book/appa.html`

The future can not be predicted

A sad Kernel Story – HURD

- 1984: Richard Stallman founds the GNU Project
- Objective: Develop a free Unix operating system
  $\Longrightarrow$ **GNU HURD**
- GNU HURD system consists of:
  - GNU Mach, the microkernel
  - File systems, protocols, servers (services), which run in user mode
  - GNU software, e.g. editors (Emacs), compilers (GCC), debugger (GDB), shell (Bash),...
- GNU HURD is *so far* complete
  - The GNU software is almost completed since the early 1990s
  - Not all servers are completely implemented
- The component that is the most challenging to implement: The microkernel

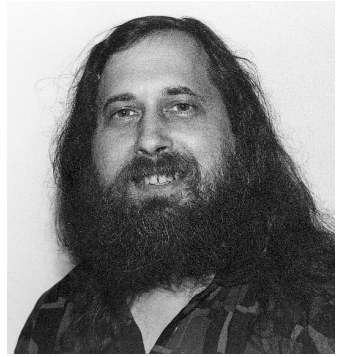
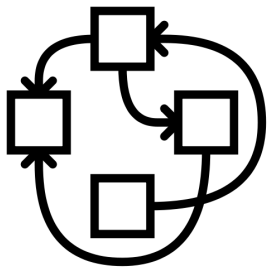


Image source: `stallman.org`



Wikipedia (CC-BY-SA-2.0)



Wikipedia (CC-BY-SA-3.0)

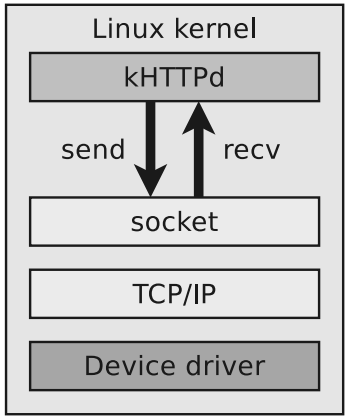
An extreme Kernel Story – kHTTPd

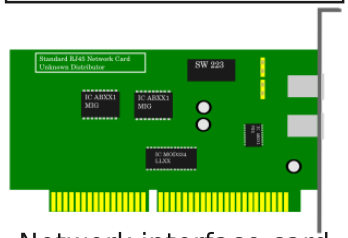
http://www.fenrus.demon.nl

- 1999: Arjan van de Ven develops the **kernel-based web server** kHTTPd for Linux kernel 2.4.x

The Design of kHTTPd: https://www.linux.it/~rubini/docs/khttpd/khttpd.html
Announce: kHTTPd 0.1.0: http://static.lwn.net/1999/0610/a/khttpd.html

- Advantage: Faster delivery of static(!) web pages
  - Less switching between user mode and kernel mode is required
- Drawback: Security risk
  - Complex software like a web server should not run in kernel mode
  - Bugs in the web server could cause system crashes or enable an attacker to takeover system control
- Linux kernel $\geq$ 2.6.x does not contain kHTTPd





Network interface card

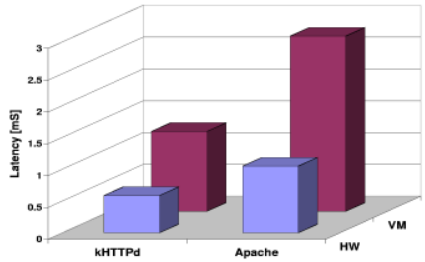
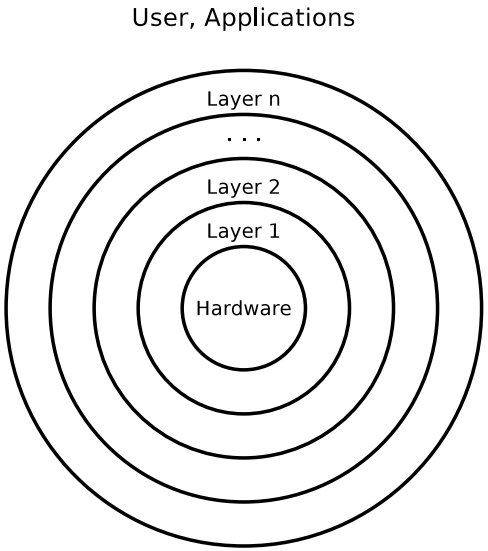


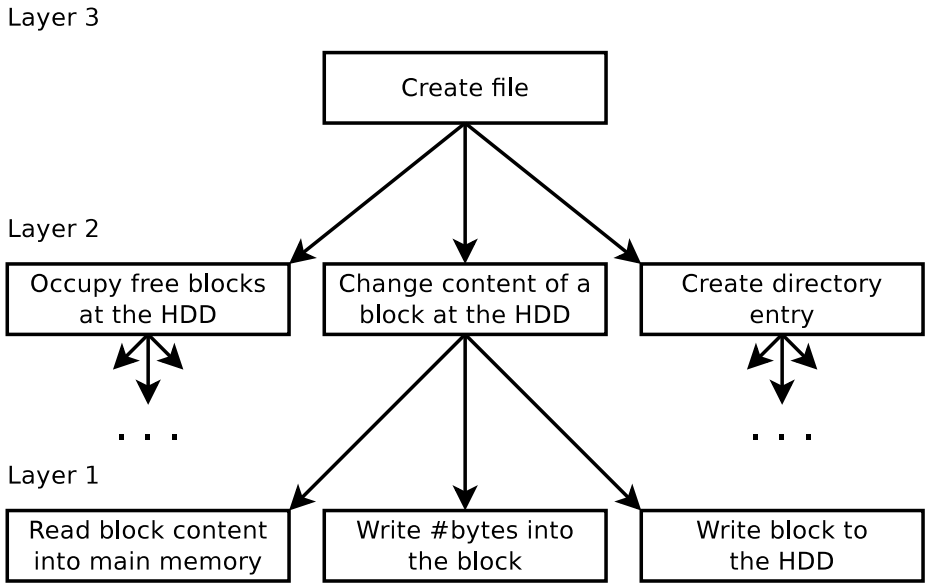
Image source:
Kernel Plugins: When A VM Is Too Much. *Ivan Ganev, Greg Eisenhauer, Karsten Schwan*. 2004

Structure (Layers) of Operating Systems (1/2)

- Operating systems can be logically structured via layers
  - The layers surround each other
  - The layers contain more and more abstract functions from the inside out
- The minimum is 3 layers:
  - The **innermost layer** contains the hardware-dependent parts of the operating system
    - This layer allows to (theoretically!) easily port operating systems to different computer architectures
  - The **central layer** contains basic input/output services (libraries and interfaces) for devices and data
  - The **outermost layer** contains the applications and the user interface
- Usually, operating systems are illustrated with more than 3 logical layers

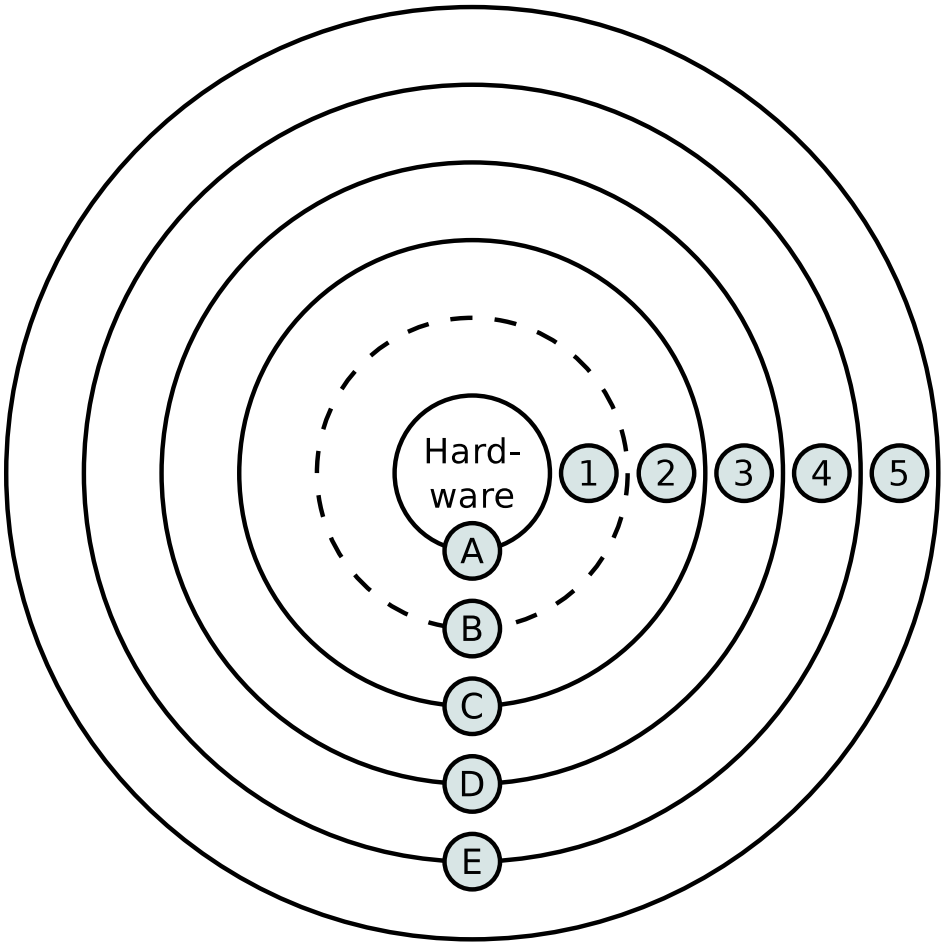
Structure (Layers) of Operating Systems (2/2)





- Each layer is similar with an **abstract machine**
- Layers communicate with neighboring layers via **well-defined interfaces**
- Layers can call functions of the next inside layer
- Layers provide functions to the next outside layer
- All functions (**services**), which are offered by a layer, and the rules, which must be observed, are called **protocol**

Layers of Linux/UNIX

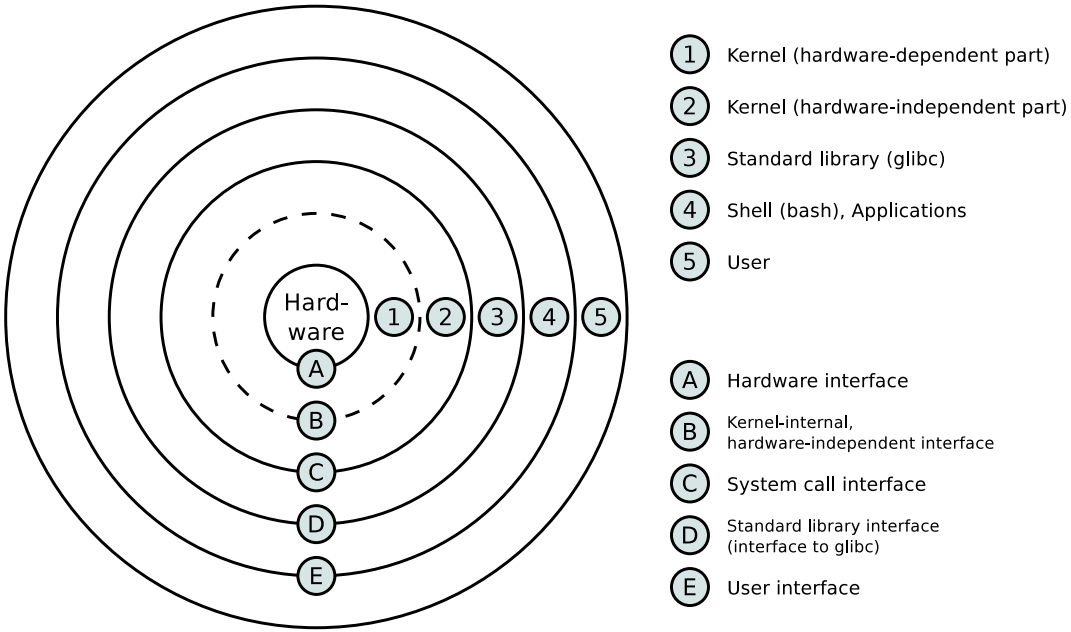


1. Kernel (hardware-dependent part)
2. Kernel (hardware-independent part)
3. Standard library (glibc)
4. Shell (bash), Applications
5. User

A. Hardware interface
B. Kernel-internal, hardware-independent interface
C. System call interface
D. Standard library interface (interface to glibc)
E. User interface

In practice, the concept is not strictly followed all the time. User applications, can e.g. call wrapper function of the standard library glibc or directly call the system calls ($\Longrightarrow$ see slide set 7)

Topics of the Operating Systems Module



- In this module, the contents of the layers are discussed
  - Layer 0 $\Longrightarrow$ **Hardware:** Slide sets 3+4
  - Layer 1 $\Longrightarrow$ **Kernel architecture:** Slide set 2
  - Layer 2 $\Longrightarrow$ **Kernel functions:** Slide sets 5+6+7+8+9
  - Layer 3 $\Longrightarrow$ **Standard library:** Slide sets 7+9
  - Layer 4 $\Longrightarrow$ **Shell:** Exercise sheets and examples on slide sets
  - Layer 5 $\Longrightarrow$ **User:** You :-)

Booting the Operating System

- The process of starting a computer and its operating system is called the **boot process**, or **bootstrapping**
  - It includes steps from initializing the computer's hardware components to passing control to the operating system and its users or their processes and providing a user interface
- The individual steps of the boot process in Linux/UNIX operating systems are:
  1. switch on the computer
  2. start the firmware and perform a self-test
  3. start the boot loader
  4. start the operating system kernel and the temporary root file system
  5. mount the real root file system
  6. start init/systemd and the system processes
  7. pass control to the users

The following slides describe the individual steps of the boot process

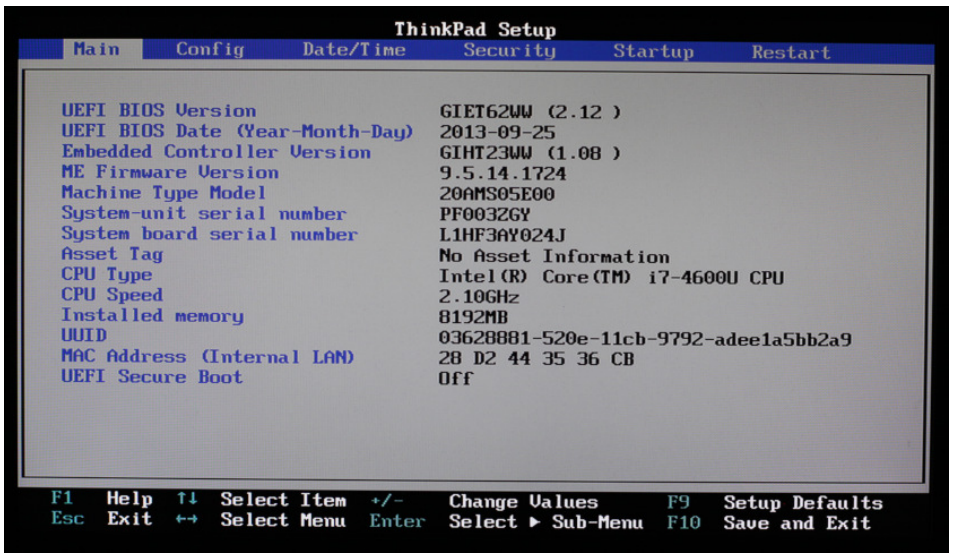
(1) Switch on the Computer

- In older computers (up to the 2000s), the mainboard is powered up when the computer is turned on and the processor starts the **Von Neumann Fetch-Decode-Execute cycle** ($\Longrightarrow$ slide set 3)
- In newer computers, an **autonomous subsystem** is usually running continuously, such as:
  - **Intel Management Engine** (since 2008)
  - **AMD Platform Security Processor** (since 2013)
  - Such subsystems are independent microcontrollers on the mainboard or in the chipset ($\Longrightarrow$ slide set 3) or alternatively special processor cores in the main processor with their own operating system
    - They usually run whenever a sufficiently charged battery or a permanent power source is present
    - They allow a computer to be monitored and woken up over the network (**Wake-on-LAN**) and provide remote administration capabilities (**remote management**)

- Carikli D. (2018). **The Intel Management Engine: an attack on computer users' freedom.** Free Software Foundation. `https://static.fsf.org/nosvn/blogs/Intel_ME_Carikli_article_PRINT_2.pdf`
- Ermolov M, Goryachy M. (2017). **How to Hack a Turned-Off Computer, or Running Unsigned Code in Intel Management Engine.** Black Hat Europe. London

(2) Start the Firmware and perform a Self-Test (1/2)

- The firmware is started when the computer is switched on.
  - Older computers with x86-compatible processors from the early 1980s to the late 2000s have a **BIOS** (Basic Input/Output System) as firmware.
  - Newer computers have a **UEFI** (Unified Extensible Firmware Interface).



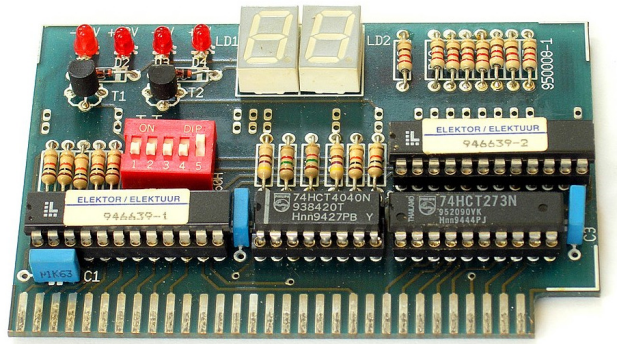
BIOS of a Thinkpad X240



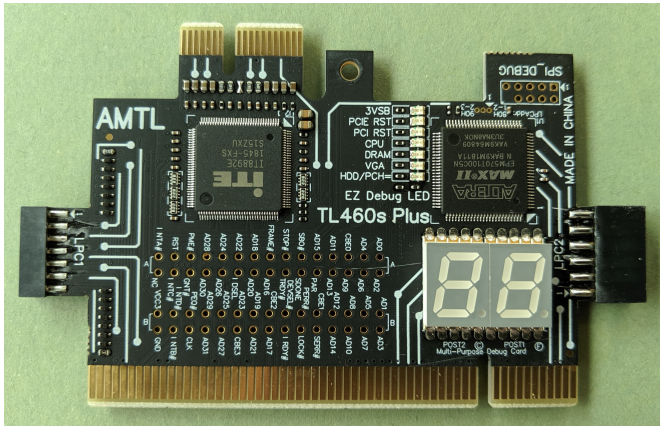
UEFI of an ASUS Z87-C

(2) Start the Firmware and perform a Self-Test (2/2)

- The firmware...
  - performs the **POST** (power-on self-test)
    - During this procedure, the CPU, cache and main memory are checked for correct operation
    - The presence of graphical output hardware, input/output devices and storage drives is also checked
- Status and error messages are indicated during the POST on the monitor or by acoustic signals (beeps)
- Most computers can be equipped with diagnostic cards (**POST cards**) to monitor the self-test process



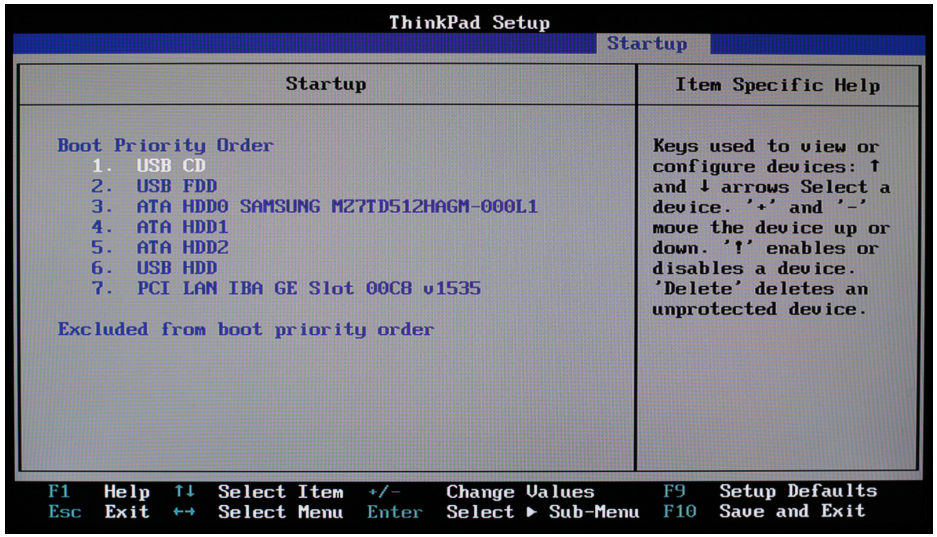
POST card for PCI
Image source: Jahoe. Wikimedia (CC0)



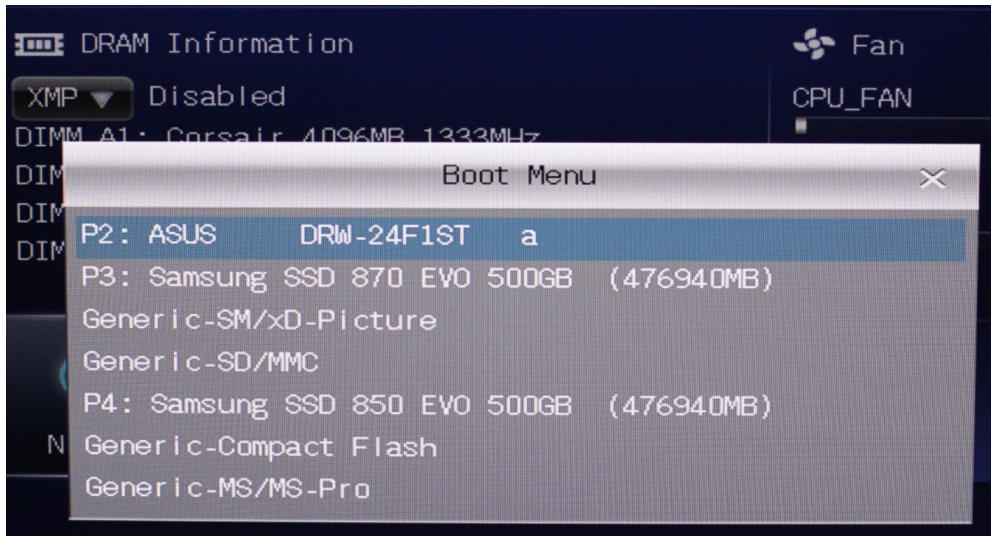
POST card for PCI, PCIe and LPC
Image source: Markus Kuhn. Wikimedia (CC0)

(3) Start the Boot Loader (1/4)

- Once the computer has been booted and passed self-test, the firmware will search for the first boot device (boot drive)
  - The device order is defined by the user in the firmware or follows the default boot order
  - The boot device can be a drive (e.g. SSD, hard disk, USB memory stick) or a network resource
    ($\Longrightarrow$ PXE protocol = **Preboot Execution Environment**)



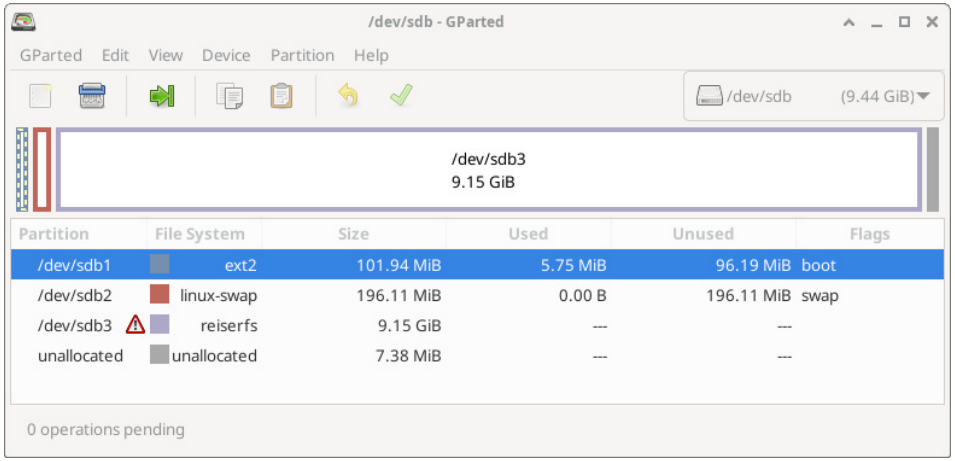
BIOS of a Thinkpad X240



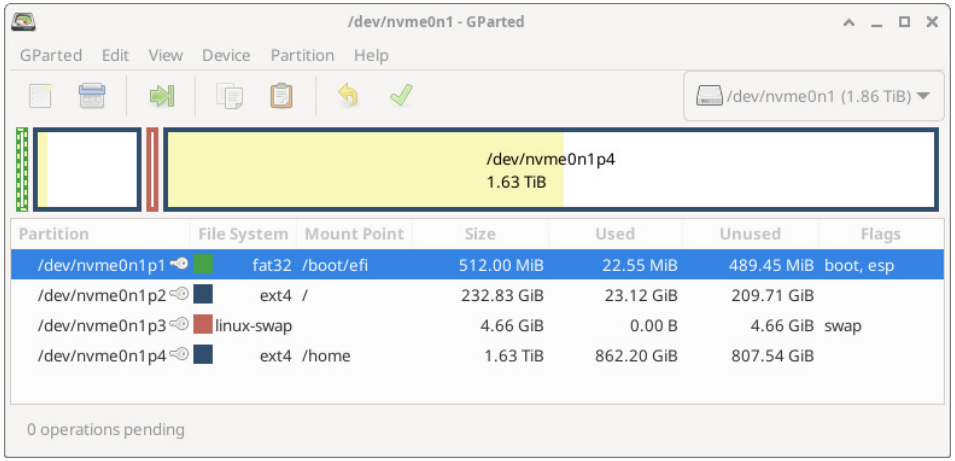
UEFI of an ASUS Z87-C

(3) Start the Boot Loader (2/4)

- The firmware starts the boot loader from the selected boot device
  - The boot loader is a program that loads the operating system
  - The location of the boot loader on the boot device depends on the **partitioning scheme** used
    - When using a classic PC partition table, the boot loader is stored in the 512 byte *master boot record* (MBR)
    - When using a GUID partition table (GPT), the boot loader is stored in the ESP (EFI System Partition)



The MBR is not visible here!



The ESP is visible here

Information about the MBR: `https://knowitlikepro.com/understanding-master-boot-record-mbr/`
Information about the ESP (P.117+118): `https://uefi.org/sites/default/files/resources/UEFI%202_5.pdf`

(3) Start the Boot Loader (3/4)

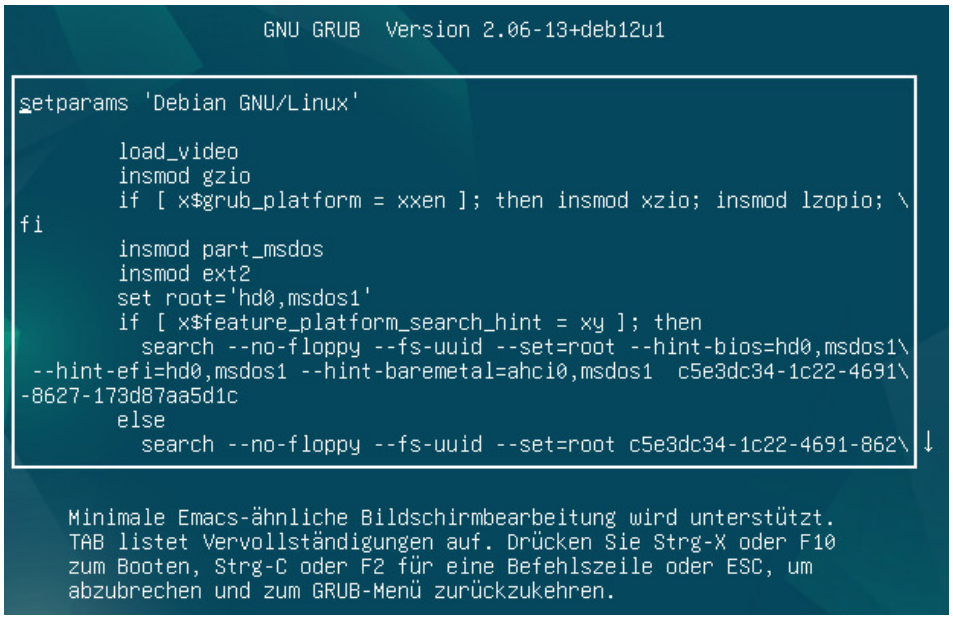
- The firmware loads the boot loader from the boot device and writes it into the main memory
  - Some popular boot loaders include: **GRUB**, **Windows Boot Manager**, **Clover**,...
- Modern boot loaders such as GRUB and Clover allow users to boot operating systems with different parameters or in a protected mode
- If there are several operating systems on the computer, modern boot loaders also allow selecting one of these operating systems

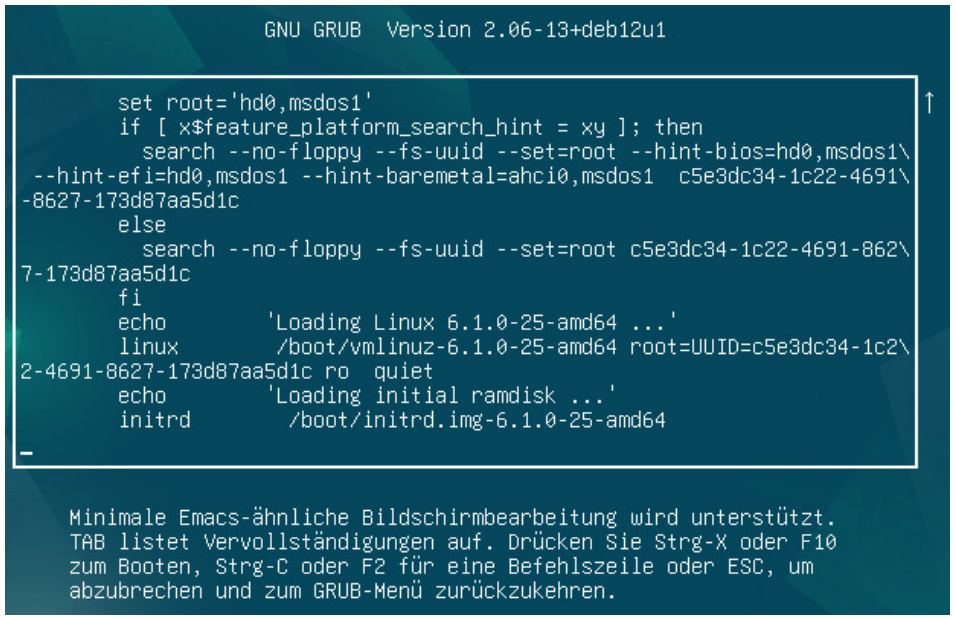




(3) Start the Boot Loader (4/4)

- GRUB even provides a command-line interpreter, the so-called GRUB shell
  - It provides information and command-line tools for repairing the GRUB boot loader configuration and manually controlling the boot process





(4) Start the Kernel and the temporary Root File System

- Once the user has manually or automatically selected an operating system or kernel using the boot loader, the kernel it unpacked and written into the main memory

In Linux, the kernel is typically a compressed file with the file name `vmlinuz-<version>-<architecture>` stored in the `/boot` folder. In the Microsoft Windows NT family, the file is called `Ntoskrnl.exe` and is stored in the `\Windows\System32` folder

- Next, the boot loader loads the initial RAM disk (`initrd`) or the initial RAM file system (`initramfs`) into the main memory
  - This is a temporary root file system loaded into the main memory
  - In Linux/UNIX, the root file system (root directory) is identified by a slash character (/)
    - The temporary root file system loaded by `initrd` or `initramfs` implements a minimal Linux environment in the main memory
    - Its primary purpose is to provide the kernel with more device drivers, file systems drivers, and programs to mount the real root file system of the operating system into the main memory

In Linux operating systems, the initial RAM disk is also usually stored as a compressed file in the `/boot` folder with the file name `initrd.img-<version>-<architecture>`

(5) Mount the real Root File System

- From within the **temporary root file system**, the kernel accesses the drive (usually an SSD or HDD) containing the **real root file system**
  - The kernel validates its consistency (correctness) of the root file system and corrects any errors in the file system
    - For this purpose, the kernel typically uses userspace tools (e.g., `e2fsck`, `xfs_repair`, `btrfs check`)
    - If necessary, the kernel can replay committed journal transactions to restore file system consistency
- The kernel then mounts the real root file system for replacing the temporary root file system
  - Furthermore, the kernel mounts any other file systems that may be available (e.g., `/home`)

(6) Start init/systemd and the System Processes

- After the kernel is started and the root file system is mounted, the kernel starts `init` as the first user-mode process
  - The process `init` has the process ID 1
  - All other user-mode processes in the system are descended from `init` ($\Longrightarrow$ slide set 7)
- At this stage of the boot process, numerous system processes and services (cron daemon, SSH server, web server, etc.) are started automatically
  - Until the late 2000s, Linux operating systems, like many other UNIX operating systems, included a System V-style implementation of `init`, also called **sysvinit**
  - Since the early 2010s, most popular Linux distributions used the more advanced **systemd**

Advantages of systemd include a faster system boot by starting system processes (services) in parallel, an integrated logging system called `journald` for unified event logging and analysis, and the ability to restart failed services automatically

(7) Pass Control to the Users (1/2)

- As the final step in the boot process, the kernel passes control to the user and their user-mode processes
  - The kernel continues to run in main memory in **kernel mode** ($\Longrightarrow$ slide set 5), managing the hardware resources and system calls ($\Longrightarrow$ slide set 7)
- The `getty` processes are also started at this stage
  - They provide a text-based login for users via one or more **(virtual) consoles**
    - The operating system starts a separate instance of the `getty` process for each of the virtual consoles (TTY1 to TTY6)

```
$ ps ax | grep getty
   1533 tty1     Ss+    0:00 /sbin/agetty -o -p -- \u --noclear - linux
  32078 tty2     Ss+    0:00 /sbin/agetty -o -p -- \u --noclear - linux
  32095 tty3     Ss+    0:00 /sbin/agetty -o -p -- \u --noclear - linux
  32098 tty4     Ss+    0:00 /sbin/agetty -o -p -- \u --noclear - linux
  32100 tty5     Ss+    0:00 /sbin/agetty -o -p -- \u --noclear - linux
  32102 tty6     Ss+    0:00 /sbin/agetty -o -p -- \u --noclear - linux
  32510 pts/9    S+     0:00 grep getty
```

- Typical Linux systems have six virtual consoles, which can be accessed using the keys `Ctrl+Alt+F1` to `Ctrl+Alt+F6`
- For historical reasons, these virtual consoles are called **TTY** (teletypewriter)

(7) Pass Control to the Users (2/2)

- In each virtual console, the operating system starts a **shell** after successful login, which provides a user interface for accessing the system via the command-line
  - Examples of shells include `bash`, `fish`, `ksh`, `csh`, `tcsh`, or `zsh`
- If the operating system uses a **graphical login manager** (e.g., GDM, LightDM, or XDM), it will also be started during this final step of the boot process too.

The graphical login manager typically runs on TTY7 and can be accessed on many Linux distributions using the `Ctrl+Alt+F7` key combination

- After logging in via the graphical login manager, the desktop environment is loaded, which provides the user with a graphical user interface
  - Examples of desktop environments include: XFCE, GNOME, KDE, Window Maker or Enlightenment